

R Programming Practice (Part 6): Loop Functions

Tim Irwin

2026-04-27

R Programming Practice (Part 6): Loop Functions

This practice set covers the loop functions from *R Programming for Data Science* by Roger D. Peng (pages 75–93). These are the functions that let you apply operations across whole data structures in one compact line, instead of writing multi-line `for` loops.

The five functions covered here, in the order they’re presented in the book:

1. `lapply()` — Loop over a list, apply a function to each element, return a list.
2. `split()` — Split a vector or data frame into groups defined by a factor.
3. `tapply()` — Apply a function over subsets of a vector (essentially `split` + `apply` combined).
4. `apply()` — Apply a function over the rows or columns of a matrix.
5. `mapply()` — Apply a function in parallel over multiple vectors.

Each section starts with a **lesson** that teaches the concept with a worked example, followed by **exercises** that apply it on built-in datasets. Sections build on each other — `split` + `lapply` lead naturally into `tapply`, and ideas from earlier sections recur in the mini challenges.

You may want to start with:

```
rm(list = ls())
```

Tip: Read the lesson, run the example code in your console to see what each function returns, then attempt the exercise. The output structure (list vs. vector vs. matrix) is often the trickiest part — if you’re unsure, run `class()` on the result.

Datasets used in this practice

- `mtcars` — Fuel consumption and 10 design/performance attributes for 32 cars.
- `iris` — Measurements of 150 iris flowers across 3 species.
- `airquality` — Daily air quality measurements in New York, May–September 1973.
- `state.x77` — A matrix of statistics on U.S. states (population, income, illiteracy, etc.).
- `ChickWeight` — Weight of chicks on different diets over time.

1. `lapply()`

Lesson 1.1: What `lapply()` does

`lapply()` does three things:

1. Loops over a list, taking each element one at a time
2. Applies a function (that you supply) to each element
3. Returns a **list** of results — always a list, regardless of what the function returns

The signature is `lapply(X, FUN, ...)`:

- `X` is the list to loop over (if it's not a list, R coerces it to one)
- `FUN` is the function to apply
- `...` is for extra arguments to pass to `FUN`

Here's a basic example. Suppose you have a list of numeric vectors and you want the mean of each:

```
my_list <- list(
  weekend_steps = c(8500, 9200, 12000),
  weekday_steps = c(6500, 7000, 7800, 6900, 7400)
)

lapply(my_list, mean)
```

```
## $weekend_steps
## [1] 9900
##
## $weekday_steps
## [1] 7120
```

Notice three things:

- We pass `mean` (without parentheses) — we're handing the function itself to `lapply()`, not calling it.
- The names from the input list (`weekend_steps`, `weekday_steps`) are preserved in the output.
- The result is a list, even though each element is a single number.

Tip: `sapply()` is `lapply()`'s simpler cousin — it tries to **simplify** the output to a vector or matrix when possible. So `sapply(my_list, mean)` would return a named numeric vector instead of a list. Use `sapply()` when the output is regular and you want it tidied up; use `lapply()` when you want the list structure preserved.

Exercise 1.1

Create a list containing three numeric vectors of different lengths (any data you like). Use `lapply()` to find the maximum value in each vector.

```
calories <- list(
  mon = c(200, 95, 725, 640, 95, 20),
  tues = c(200, 95, 480, 520, 315),
  wed = c(200, 95, 900, 720)
)

lapply(calories, max)
```

```
## $mon
## [1] 725
##
## $tues
## [1] 520
##
## $wed
## [1] 900
```

Lesson 1.2: Passing extra arguments through ...

What if the function you're applying needs additional arguments? For example, `mean()` has an `na.rm` argument. You can pass it through `lapply()`'s `...` argument:

```

incomes <- list(
  jan = c(3200, NA, 4100, 3800),
  feb = c(3300, 4200, NA, 3900),
  mar = c(3400, 4300, 4000, NA)
)

# Without na.rm, NAs propagate
lapply(incomes, mean)

## $jan
## [1] NA
##
## $feb
## [1] NA
##
## $mar
## [1] NA

# With na.rm passed through ...
lapply(incomes, mean, na.rm = TRUE)

## $jan
## [1] 3700
##
## $feb
## [1] 3800
##
## $mar
## [1] 3900

```

Any named arguments you supply after the function get forwarded to the function each time it's called.

Exercise 1.2

The `airquality` dataset has missing values in `Ozone` and `Solar.R`. Build a list containing these two columns, then use `lapply()` with `mean` and `na.rm = TRUE` to get the mean of each, ignoring NAs.

```

airquality_mean <- list(
  Ozone = airquality$Ozone,
  Solar.R = airquality$Solar.R
)

lapply(airquality_mean, mean, na.rm = T)

## $Ozone
## [1] 42.12931
##
## $Solar.R
## [1] 185.9315

```

Lesson 1.3: Anonymous functions

Sometimes the function you want to apply isn't built-in and isn't worth defining separately. You can create an **anonymous function** right inside the `lapply()` call:

```
test_scores <- list(
  alice = c(82, 91, 78, 88),
  bob = c(74, 68, 71, 80),
  carol = c(95, 92, 89, 94)
)

# Anonymous function: range of each person's scores (max - min)
lapply(test_scores, function(s) max(s) - min(s))

## $alice
## [1] 13
##
## $bob
## [1] 12
##
## $carol
## [1] 6
```

The `function(s) max(s) - min(s)` part is a function with no name — it exists only for this one call. Inside the function body, `s` refers to whichever element of the list we're currently on.

Tip: Anonymous functions are great for one-off operations. If the logic gets long or you'll reuse it, define a named function separately.

Exercise 1.3

Use `lapply()` with an anonymous function to compute the **range** (max minus min) of each numeric column of `mtcars`. Hint: a data frame *is* a list of columns, so you can pass `mtcars` directly to `lapply()`.

```
lapply(mtcars, function(s) max(s) - min(s))

## $mpg
## [1] 23.5
##
## $cyl
## [1] 4
##
## $disp
## [1] 400.9
##
## $hp
## [1] 283
##
## $drat
## [1] 2.17
##
## $wt
## [1] 3.911
##
## $qsec
## [1] 8.4
##
## $vs
## [1] 1
##
## $am
```

```
## [1] 1
##
## $gear
## [1] 2
##
## $carb
## [1] 7
```

Lesson 1.4: lapply() over a sequence

A common trick: you can loop over a sequence of integers and use the iterator as input to a function. For example, generating samples of different sizes:

```
# Generate samples of size 2, 4, and 6 from values 1-100
sample_sizes <- c(2, 4, 6)
lapply(sample_sizes, sample, x = 1:100)
```

```
## [[1]]
## [1] 50 63
##
## [[2]]
## [1] 100 28 1 78
##
## [[3]]
## [1] 79 73 15 87 20 10
```

Here `sample()` gets called three times — once with `size = 2`, once with `size = 4`, once with `size = 6`. The values from `sample_sizes` get passed as the *first* unnamed argument to `sample()`, which is `size`. We then pass `x = 1:100` through ... so each call samples from the same vector.

Exercise 1.4

Use `lapply()` to extract the first 3 rows of `mtcars`, `iris`, and `airquality` (a list of three data frames). Hint: if you put the three data frames into a list, you can apply `head()` with `n = 3` to each.

```
data <- list(
  mtcars,
  iris,
  airquality
)

lapply(data, head, n = 3)
```

```
## [[1]]
##           mpg cyl disp  hp drat   wt  qsec vs am gear carb
## Mazda RX4      21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag  21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710     22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
##
## [[2]]
##      Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1           5.1         3.5          1.4         0.2  setosa
## 2           4.9         3.0          1.4         0.2  setosa
## 3           4.7         3.2          1.3         0.2  setosa
##
```

```
## [[3]]
##   Ozone Solar.R Wind Temp Month Day
## 1    41     190  7.4   67     5   1
## 2    36     118  8.0   72     5   2
## 3    12     149 12.6   74     5   3
```

2. split()

Lesson 2.1: What split() does

split() divides a vector or data frame into groups based on a factor. It returns a **list** where each element is one group's data.

The signature is split(x, f):

- x is what you want to split (vector or data frame)
- f is the factor (or coerced to one) defining the groups

Here's an example. The iris dataset has a Species column with three levels — let's split the dataset by species:

```
iris_by_species <- split(iris, iris$Species)

# What we get back is a list of three data frames
class(iris_by_species)
```

```
## [1] "list"
```

```
names(iris_by_species)
```

```
## [1] "setosa"      "versicolor" "virginica"
```

```
# Each element is the rows for one species
head(iris_by_species$setosa, 3)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1         5.1         3.5         1.4         0.2  setosa
## 2         4.9         3.0         1.4         0.2  setosa
## 3         4.7         3.2         1.3         0.2  setosa
```

Each element of the resulting list is a complete data frame — just filtered down to one species.

You can also split a single vector by a factor:

```
# Split the Sepal.Length values by species
sepals_split <- split(iris$Sepal.Length, iris$Species)
sepals_split
```

```
## $setosa
## [1] 5.1 4.9 4.7 4.6 5.0 5.4 4.6 5.0 4.4 4.9 5.4 4.8 4.8 4.3 5.8 5.7 5.4 5.1 5.7
## [20] 5.1 5.4 5.1 4.6 5.1 4.8 5.0 5.0 5.2 5.2 4.7 4.8 5.4 5.2 5.5 4.9 5.0 5.5 4.9
## [39] 4.4 5.1 5.0 4.5 4.4 5.0 5.1 4.8 5.1 4.6 5.3 5.0
##
## $versicolor
## [1] 7.0 6.4 6.9 5.5 6.5 5.7 6.3 4.9 6.6 5.2 5.0 5.9 6.0 6.1 5.6 6.7 5.6 5.8 6.2
## [20] 5.6 5.9 6.1 6.3 6.1 6.4 6.6 6.8 6.7 6.0 5.7 5.5 5.5 5.8 6.0 5.4 6.0 6.7 6.3
## [39] 5.6 5.5 5.5 6.1 5.8 5.0 5.6 5.7 5.7 6.2 5.1 5.7
##
```

```
## $virginica
## [1] 6.3 5.8 7.1 6.3 6.5 7.6 4.9 7.3 6.7 7.2 6.5 6.4 6.8 5.7 5.8 6.4 6.5 7.7 7.7
## [20] 6.0 6.9 5.6 7.7 6.3 6.7 7.2 6.2 6.1 6.4 7.2 7.4 7.9 6.4 6.3 6.1 7.7 6.3 6.4
## [39] 6.0 6.9 6.7 6.9 5.8 6.8 6.7 6.7 6.3 6.5 6.2 5.9
```

The result is a list of three numeric vectors — one per species.

Exercise 2.1

Use `split()` to divide the `mtcars` dataset into groups based on the number of cylinders (`cyl`). The result should be a list with three elements (4-cylinder, 6-cylinder, and 8-cylinder cars). Confirm the structure with `names()` on the result.

```
cyl_mtcars <- split(mtcars, mtcars$cyl)
names(cyl_mtcars)
```

```
## [1] "4" "6" "8"
```

Lesson 2.2: The split + lapply pattern

The real power of `split()` shows up when you combine it with `lapply()`. Once you’ve split data into groups, you can apply a function to each group:

```
# Step 1: split
sepals_by_species <- split(iris$Sepal.Length, iris$Species)
```

```
# Step 2: apply
lapply(sepals_by_species, mean)
```

```
## $setosa
## [1] 5.006
##
## $versicolor
## [1] 5.936
##
## $virginica
## [1] 6.588
```

Or, more compactly, in a single line:

```
lapply(split(iris$Sepal.Length, iris$Species), mean)
```

```
## $setosa
## [1] 5.006
##
## $versicolor
## [1] 5.936
##
## $virginica
## [1] 6.588
```

Read this from the inside out: first split the vector by species, then apply `mean` to each piece.

Tip: “Split, apply, combine” is one of the fundamental patterns in data analysis. R has a built-in shortcut for the vector case (`tapply()`, coming next) and entire packages (`dplyr`, `data.table`) built around making it more convenient.

Exercise 2.2

Using your split from Exercise 2.1, use `lapply()` to compute the **average horsepower (hp)** for cars in each cylinder group. Hint: each element of your split is a full data frame, so your function will need to extract the `hp` column from it.

```
lapply(split(mtcars$hp, mtcars$cyl), mean)
```

```
## $'4'
## [1] 82.63636
##
## $'6'
## [1] 122.2857
##
## $'8'
## [1] 209.2143
```

3. `tapply()`

Lesson 3.1: What `tapply()` does

`tapply()` is essentially `split()` + `sapply()` rolled into one, but only for vectors. If you find yourself doing `lapply(split(x, f), some_function)` often, `tapply()` is the shortcut.

The signature is `tapply(X, INDEX, FUN, ...)`:

- `X` is the vector
- `INDEX` is the grouping factor
- `FUN` is the function to apply
- `...` is for extra arguments to `FUN`

Here's the same calculation we did with `split` + `lapply`, but in one call:

```
# Mean Sepal.Length per species, the lapply way:
lapply(split(iris$Sepal.Length, iris$Species), mean)
```

```
## $setosa
## [1] 5.006
##
## $versicolor
## [1] 5.936
##
## $virginica
## [1] 6.588
```

```
# The tapply way:
tapply(iris$Sepal.Length, iris$Species, mean)
```

```
##      setosa versicolor  virginica
##      5.006      5.936      6.588
```

Notice the output difference: `tapply()` simplifies the result to a named numeric vector by default, while `lapply()` returns a list. When the function returns a single number per group, this is much cleaner.

Exercise 3.1

Using `tapply()`, compute the average mpg for each cylinder group in `mtcars`. The result should be a named vector showing average mpg for 4-, 6-, and 8-cylinder cars.

```
tapply(mtcars$mpg, mtcars$cyl, mean)
```

```
##           4           6           8
## 26.66364 19.74286 15.10000
```

Exercise 3.2

Using `tapply()` on the `airquality` dataset, compute the average Temp for each Month. Make sure to handle NAs by passing `na.rm = TRUE`.

```
tapply(airquality$Temp, airquality$Month, mean, na.rm = T)
```

```
##           5           6           7           8           9
## 65.54839 79.10000 83.90323 83.96774 76.90000
```

Lesson 3.2: When the function returns more than one value

If the function returns multiple values per group, `tapply()` can't simplify to a vector and returns a list instead:

```
# range() returns two values: min and max
tapply(iris$Sepal.Length, iris$Species, range)
```

```
## $setosa
## [1] 4.3 5.8
##
## $versicolor
## [1] 4.9 7.0
##
## $virginica
## [1] 4.9 7.9
```

Each list element is a length-2 vector containing the min and max for that species.

Exercise 3.3

Using `tapply()` on `ChickWeight`, get the **range** of weights for each diet (the `Diet` column). The result will be a list since `range()` returns two values per group.

```
tapply(ChickWeight$weight, ChickWeight$Diet, range)
```

```
## $'1'
## [1] 35 305
##
## $'2'
## [1] 39 331
##
## $'3'
## [1] 39 373
##
## $'4'
## [1] 39 322
```

4. apply()

Lesson 4.1: What apply() does

`apply()` is for **matrices and arrays** — it applies a function over the rows or columns. It's not really about looping over a list; it's about computing something for every row or column at once.

The signature is `apply(X, MARGIN, FUN, ...)`:

- `X` is the matrix (or array)
- `MARGIN` is 1 for rows, 2 for columns
- `FUN` is the function to apply
- `...` is for extra arguments

Here's the trick that confuses many people: the `MARGIN` argument tells `apply()` which dimension to **preserve**, not which to iterate over. So `MARGIN = 2` means “preserve the columns” — which means apply the function to each column, collapsing the rows down to one value per column.

`state.x77` is a built-in matrix of state statistics. Let's compute the mean of each column:

```
class(state.x77)      # confirm it's a matrix

## [1] "matrix" "array"

dim(state.x77)         # 50 states × 8 statistics

## [1] 50  8

# Mean of each column (preserve columns, collapse rows)
apply(state.x77, 2, mean)
```

```
## Population      Income Illiteracy   Life Exp      Murder    HS Grad      Frost
## 4246.4200 4435.8000      1.1700    70.8786    7.3780    53.1080    104.4600
##      Area
## 70735.8800
```

Now the mean of each row (mostly nonsensical here, but illustrative):

```
# First few rows only, to keep output short
apply(state.x77[1:5, ], 1, mean)

##      Alabama      Alaska      Arizona      Arkansas California
## 7261.819 71676.601 15039.031 7202.570 22854.839
```

Tip: For row/column means and sums, R has shortcuts: `rowMeans()`, `colMeans()`, `rowSums()`, `colSums()`. They're faster and more readable than `apply()` for those specific operations. Use `apply()` when you need a different function (median, sd, custom logic, etc.).

Exercise 4.1

Use `apply()` on `state.x77` to find the **maximum value** in each column. The result should be a named numeric vector showing the highest population, highest income, etc.

```
apply(state.x77, 2, max)

## Population      Income Illiteracy   Life Exp      Murder    HS Grad      Frost
## 21198.0 6315.0      2.8      73.6      15.1      67.3      188.0
##      Area
## 566432.0
```

Exercise 4.2

Convert the first 5 rows of `mtcars` to a matrix using `as.matrix()`, then use `apply()` to compute the **standard deviation** of each column. (The SD function is `sd()`.)

```
car_matrix <- as.matrix(head(mtcars, n = 5))
```

```
apply(car_matrix, 2, sd)
```

```
##      mpg      cyl      disp      hp      drat      wt
## 1.4737707 1.4142136 100.2556732 31.8323735 0.4220545 0.4489070
##      qsec      vs      am      gear      carb
## 1.2567418 0.5477226 0.5477226 0.5477226 1.5165751
```

Lesson 4.2: Passing extra arguments

Just like `lapply()`, `apply()` accepts extra arguments via `...` that get forwarded to the function. A common case is using `quantile()` to get specific percentiles:

```
# Get the 25th and 75th percentile of each column of state.x77
```

```
apply(state.x77, 2, quantile, probs = c(0.25, 0.75))
```

```
##      Population Income Illiteracy Life Exp Murder HS Grad Frost      Area
## 25%      1079.5 3992.75      0.625 70.1175 4.350 48.05 66.25 36985.25
## 75%      4968.5 4813.50      1.575 71.8925 10.675 59.15 139.75 81162.50
```

Here `probs = c(0.25, 0.75)` gets passed to `quantile()` each time it's called. The result is a matrix where each column has two values (the two percentiles).

Exercise 4.3

Using `apply()` on `state.x77`, get the **5th and 95th percentile** of each column. Hint: it's the same pattern as the lesson example, just different `probs` values.

```
apply(state.x77, 2, quantile, probs = c(0.05, 0.95))
```

```
##      Population Income Illiteracy Life Exp Murder HS Grad Frost      Area
## 5%      520.15 3570.2      0.545 68.639 2.300 39.130 13.35 5565.35
## 95%     12067.35 5271.1      2.255 72.846 12.885 64.615 173.55 151512.70
```

Lesson 4.3: Anonymous functions with `apply()`

When you need custom logic, anonymous functions work the same way as with `lapply()`:

```
# For each column, compute the ratio of max to min
```

```
apply(state.x77, 2, function(col) max(col) / min(col))
```

```
## Population      Income Illiteracy Life Exp      Murder      HS Grad      Frost
## 58.076712    2.038412    5.600000    1.082990    10.785714    1.780423      Inf
##      Area
## 539.973308
```

This shows you how spread out each variable is — Population has a much larger ratio than, say, Life Exp, because state populations vary much more than life expectancy.

Exercise 4.4

Using `apply()` with an anonymous function, compute the **range** (max minus min) of each column of `state.x77`.

```
apply(state.x77, 2, function(col) max(col) - min(col))
```

```
## Population      Income Illiteracy   Life Exp      Murder    HS Grad      Frost
##   20833.00    3217.00        2.30      5.64      13.70     29.50     188.00
##      Area
##  565383.00
```

5. mapply()

Lesson 5.1: What `mapply()` does

`mapply()` is the **multivariate** version of `lapply()`. While `lapply()` iterates over one list, `mapply()` iterates over **multiple** lists or vectors **in parallel**, applying a function to corresponding elements.

The signature is different from the others — the function comes first:

```
mapply(FUN, ..., MoreArgs = NULL, SIMPLIFY = TRUE)
```

- `FUN` is the function to apply
- `...` contains the vectors/lists to iterate over (one per function argument)
- `MoreArgs` is a list of extra arguments shared across all calls
- `SIMPLIFY` controls whether the result gets simplified (like `sapply()`)

Here's a simple example. Suppose I want to compute the BMI for several people, each with their own weight and height:

```
weights <- c(150, 165, 180, 200) # pounds
heights <- c(65, 70, 68, 72)    # inches

# Imperial BMI formula
bmi <- function(w, h) {
  (w / h^2) * 703
}

mapply(bmi, weights, heights)
```

```
## [1] 24.95858 23.67245 27.36592 27.12191
```

`mapply()` calls `bmi()` four times: `bmi(150, 65)`, `bmi(165, 70)`, `bmi(180, 68)`, `bmi(200, 72)`. The first vector feeds the first argument, the second vector feeds the second argument, etc.

Note: This particular case could be done with vectorized arithmetic: `(weights / heights^2) * 703`. `mapply()` is most useful when the function isn't already vectorized — that is, when calling it on the whole vector wouldn't do what you want.

Exercise 5.1

Write a function `pct_change(old, new)` that returns the percent change from `old` to `new`: `((new - old) / old) * 100`. Then use `mapply()` to apply it to two vectors:

```
old_prices <- c(50, 75, 100, 200)
new_prices <- c(55, 70, 110, 195)
```

The result should be a vector of percent changes for each pair.

```
old_prices <- c(50, 75, 100, 200)
new_prices <- c(55, 70, 110, 195)

pct_change <- function(old, new) {
  ((new - old) / old * 100)
}

mapply(pct_change, old_prices, new_prices)

## [1] 10.000000 -6.666667 10.000000 -2.500000
```

Lesson 5.2: Using MoreArgs for shared parameters

When all calls should use the same extra argument, use `MoreArgs`:

```
# Function: round a number to a given number of decimal places
my_round <- function(x, digits) {
  round(x, digits)
}

values <- c(3.14159, 2.71828, 1.41421)
decimals <- c(2, 3, 4)

# Each value gets rounded to its corresponding decimal place
mapply(my_round, values, decimals)

## [1] 3.1400 2.7180 1.4142
```

If instead you wanted to round all three values to the *same* number of decimals, you'd put it in `MoreArgs`:

```
mapply(my_round, values, MoreArgs = list(digits = 2))

## [1] 3.14 2.72 1.41
```

Exercise 5.2

Write a function `weighted_score(score, weight)` that returns `score * weight`. Use `mapply()` to apply it to a vector of scores `c(85, 92, 78, 88)` paired with weights `c(0.2, 0.3, 0.2, 0.3)`.

```
weighted_score <- function(score, weight) {
  score * weight
}

scores <- c(85, 92, 78, 88)
weights <- c(0.2, 0.3, 0.2, 0.3)

mapply(weighted_score, scores, weights)

## [1] 17.0 27.6 15.6 26.4
```

Lesson 5.3: Vectorizing a non-vectorized function

A nice use of `mapply()` is to “vectorize” a function that wasn’t designed to handle vectors. Suppose you wrote this earlier:

```

classify <- function(x) {
  if (is.na(x)) {
    "missing"
  } else if (x > 0) {
    "positive"
  } else if (x < 0) {
    "negative"
  } else {
    "zero"
  }
}

```

This works on one number at a time. If you call `classify(c(-1, 0, 1))`, the `if` statement fires a warning and only uses the first element — not what you want. You can wrap it with `mapply()` to apply it element-by-element:

```
mapply(classify, c(-5, 0, 3, NA, 7))
```

```
## [1] "negative" "zero"      "positive" "missing"  "positive"
```

This works because `mapply()` calls `classify()` separately on each value.

Exercise 5.3

Write a function `temp_label(temp)` that returns "hot" if `temp > 80`, "cold" if `temp < 60`, and "mild" otherwise. Use `mapply()` to apply it to `airquality$Temp` (just the first 10 values is fine).

```

temp_label <- function(temp) {
  if (temp > 80) {
    "hot"
  } else if (temp < 60) {
    "cold"
  } else {
    "mild"
  }
}

```

```
mapply(temp_label, head(airquality$Temp, n = 10))
```

```
## [1] "mild" "mild" "mild" "mild" "cold" "mild" "mild" "cold" "mild" "mild"
```

Mini Challenges

These tie everything together — each one combines multiple loop functions or layers them in interesting ways.

Challenge 1: Group means three ways

Compute the average `Petal.Length` for each species in `iris` three different ways:

1. Using `split()` + `lapply()`
2. Using `tapply()`
3. Using a `for` loop (just to remind yourself what these functions are saving you from)

Confirm all three approaches give the same numeric values.

```

# Your code here
lapply(split(iris$Petal.Length, iris$Species), mean)

## $setosa
## [1] 1.462
##
## $versicolor
## [1] 4.26
##
## $virginica
## [1] 5.552

tapply(iris$Petal.Length, iris$Species, mean)

##      setosa versicolor  virginica
##      1.462      4.260      5.552

results <- list()
for (sp in unique(iris$Species)) {
  results[[sp]] <- mean(iris$Petal.Length[iris$Species == sp])
}
results

## $setosa
## [1] 1.462
##
## $versicolor
## [1] 4.26
##
## $virginica
## [1] 5.552

#### i got 1 and 2 right, but initially i did:
# avg_petal <- function(df, col) {
#   for df
# }
#
# then:
# avg_petal <-
#   for (unique(iris$species)) {
#     results <-
#   }
# }
#
# then:
# results <- list()
# for (sp in unique(iris$Species)) {
#   results[[sp]] <- mean(iris$Petal.Length)
# }
# results

```

Challenge 2: Multi-stat summary by group

Using the `airquality` dataset, compute a **multi-statistic summary** (mean, median, and max) of `Wind` for each month. Use `tapply()` with an anonymous function that returns a named vector of the three statistics.

The result will be a list (since the function returns multiple values per group).

```
# what its looking for
tapply(airquality$Wind, airquality$Month, function(x) {
  c(mean = mean(x), median = median(x), max = max(x))
})
```

```
## $'5'
##      mean   median      max
## 11.62258 11.50000 20.10000
##
## $'6'
##      mean   median      max
## 10.26667  9.70000 20.70000
##
## $'7'
##      mean   median      max
##  8.941935  8.600000 14.900000
##
## $'8'
##      mean   median      max
##  8.793548  8.600000 15.500000
##
## $'9'
##      mean median      max
##   10.18  10.30  16.60
```

```
# this was my initial answer
#tapply(airquality$Wind, airquality$Month, summary)
```

Challenge 3: Custom summary across columns

Use `apply()` (or `lapply()` — your choice, but think about which output structure you’d prefer) to compute, for each numeric column of `mtcars`, a named vector with: minimum, mean, and maximum. The output should let you easily see “for `mpg`, min is X, mean is Y, max is Z.”

Hint: write a small helper function that takes a vector and returns the three stats as a named vector, then apply it across columns.

```
stats <- function(x) {
  c(min = min(x), mean = mean(x), max = max(x))
}
```

```
lapply(mtcars, stats)
```

```
## $mpg
##      min      mean      max
## 10.40000 20.09062 33.90000
##
## $cyl
##      min      mean      max
##  4.00000  6.18750  8.00000
##
## $disp
##      min      mean      max
```



```
## 71.1000 230.7219 472.0000
##
## $hp
##      min      mean      max
## 52.0000 146.6875 335.0000
##
## $drat
##      min      mean      max
## 2.760000 3.596563 4.930000
##
## $wt
##      min      mean      max
## 1.51300 3.21725 5.42400
##
## $qsec
##      min      mean      max
## 14.50000 17.84875 22.90000
##
## $vs
##      min      mean      max
## 0.00000 0.4375 1.0000
##
## $am
##      min      mean      max
## 0.00000 0.40625 1.00000
##
## $gear
##      min      mean      max
## 3.0000 3.6875 5.0000
##
## $carb
##      min      mean      max
## 1.0000 2.8125 8.0000
```

```
apply(mtcars, 2, stats)
```

```
##      mpg      cyl      disp      hp      drat      wt      qsec      vs      am
## min 10.40000 4.0000 71.1000 52.0000 2.760000 1.51300 14.50000 0.0000 0.00000
## mean 20.09062 6.1875 230.7219 146.6875 3.596563 3.21725 17.84875 0.4375 0.40625
## max 33.90000 8.0000 472.0000 335.0000 4.930000 5.42400 22.90000 1.0000 1.00000
##      gear      carb
## min 3.0000 1.0000
## mean 3.6875 2.8125
## max 5.0000 8.0000
```

```
# got both of these right
```

Challenge 4: ChickWeight analysis

The `ChickWeight` dataset records chick weights over time under different diets. Use loop functions to answer:

1. What is the average weight of chicks on each Diet? (Use `tapply()`.)
2. Split the dataset by `Diet` into a list of four data frames. (Use `split()`.)
3. For each diet's data frame, compute the **maximum weight** observed. (Use `lapply()` on the result of

step 2 with an anonymous function that extracts and maxes the weight column.)

```
tapply(ChickWeight$weight, ChickWeight$Diet, mean) # got right
```

```
##          1          2          3          4
## 102.6455 122.6167 142.9500 135.2627
```

```
diet <- split(ChickWeight, ChickWeight$Diet) # got right
# 3 got wrong:
# lapply(diet, function(mean(ChickWeight$weight))
lapply(diet, function(x) max(x$weight))
```

```
## $'1'
## [1] 305
##
## $'2'
## [1] 331
##
## $'3'
## [1] 373
##
## $'4'
## [1] 322
```

Challenge 5: Combining mapply() with real data

You're given two vectors — thresholds and column names:

```
thresholds <- c(20, 100, 3)
columns <- c("mpg", "hp", "wt")
```

For each pair (threshold, column), compute the **number of cars in mtcars** where that column's value exceeds the threshold. Use `mapply()` and an anonymous function.

The result should be a named vector with three numbers: how many cars have `mpg > 20`, how many have `hp > 100`, and how many have `wt > 3`.

```
thresholds <- c(20, 100, 3)
columns <- c("mpg", "hp", "wt")
```

```
mapply(function(col, thresh) sum(mtcars[[col]] > thresh), columns, thresholds) # got wrong
```

```
## mpg  hp  wt
##  14  23  20
```

```
# started with:
# car_numbers <- function(threshold, column) {
#   if (thresh)
# }
#
# then
# mapply(sum(x > threshold), mtcars[[col]])
```